

```

import matplotlib.pyplot as plt
import numpy as np
import math
import random
from time import time

n = np.arange(1, 16)

f_1 = np.ones_like(n)
f_n = n
f_log_n = np.log2(n)
f_n_log_n = n * np.log2(n)
f_n2 = n**2
f_n3 = n**3
f_2n = 2**n
f_fact = [math.factorial(x) for x in n]

functions = {
    "O(1) - Constant": f_1,
    "O(n) - Linear": f_n,
    "O(log n) - Logarithmic": f_log_n,
    "O(n log n) - Linearithmic": f_n_log_n,
    "O(n^2) - Quadratic": f_n2,
    "O(n^3) - Cubic": f_n3,
    "O(2^n) - Exponential": f_2n,
    "O(n!) - Factorial": f_fact
}

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
fig.suptitle('Individual Function Growths', fontsize=16)

for ax, (name, y_values) in zip(axes.flatten(), functions.items()):
    ax.plot(n, y_values, marker='o', color='blue')
    ax.set_title(name)
    ax.set_xlabel('Number of inputs (n)')
    ax.set_ylabel('F(n)')
    ax.grid(True)

plt.tight_layout()
plt.show()

```

```

plt.figure(figsize=(10, 6))

for name, y_values in functions.items():
    plt.plot(n, y_values, marker='o', label=name)

plt.ylim(0, 1000)
plt.xlim(1, 15)

plt.title('Overlay of All Functions (Y-axis limited to 1000 for
visibility)')
plt.xlabel('Number of inputs (n)')
plt.ylabel('F(n)')
plt.legend()
plt.grid(True)
plt.show()

```

```

# =====
# PART 2: Sorting Algorithm Runtime Analysis
# =====

```

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap elements
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

input_sizes = [100, 500, 1000, 2000, 4000, 6000, 8000, 10000]
elapsed_times = []

print("Running sorting algorithm. This may take a minute...")

```

```

for N in input_sizes:
    random_list = [random.randint(1, 10000) for _ in range(N)]

    start_time = time()
    bubble_sort(random_list)
    end_time = time()

    elapsed = end_time - start_time
    elapsed_times.append(elapsed)

    print(f"Size: {N:5d} | Elapsed Time: {elapsed:.4f} seconds")
sizes_array = np.array(input_sizes)
theoretical_n2 = sizes_array ** 2

scale_factor = elapsed_times[-1] / theoretical_n2[-1]
scaled_theoretical_n2 = theoretical_n2 * scale_factor

plt.figure(figsize=(10, 6))

plt.scatter(input_sizes, elapsed_times, color='red', zorder=5,
            label='Empirical Runtime')

plt.plot(input_sizes, scaled_theoretical_n2, color='blue', linestyle='--',
         label='Theoretical O(n^2) Growth')

plt.yscale('log')
plt.xscale('log')

plt.title('Order of Growth: Bubble Sort Runtime (Log-Log Scale)')
plt.xlabel('Input Size (N)')
plt.ylabel('Elapsed Time (Seconds)')
plt.legend()
plt.grid(True, which="both", ls="--")

plt.show()

```

Individual Function Growths



